CCS334 – BIG DATA ANALYTICS

LABORATORY PRACTICAL RECORD / TEST PREPARATION

Regulation 2021

Course	CCS334 – Big Data Analytics
Regulation	2021
Source	Uploaded CCS334 practical question paper
Questions included	1–11, 13, 14 and 16
Questions omitted	12, 15 and 17 (as requested)
Answer format	Aim • Procedure • Program (if necessary) • Output • Result

Prepared as an exam-oriented guide. Commands and sample outputs are illustrative; replace paths, usernames, hostnames and version numbers with those configured in the laboratory.

QUESTION 1: Downloading and installing Hadoop framework; startup scripts and configuration files

Aim

To install the Hadoop framework, configure it for pseudo-distributed operation, and start the Hadoop daemons using the startup scripts.

Procedure

1. Install a supported Java Development Kit and verify it using **java -version**. Set JAVA_HOME to the JDK installation directory.

2. Download a Hadoop 3.x binary distribution from the Apache Hadoop website and extract it, for example, into /usr/local/hadoop.

3. Set HADOOP_HOME and add Hadoop's bin and sbin directories to PATH. Set JAVA_HOME in Hadoop's hadoop-env.sh.

4. Configure core-site.xml, hdfs-site.xml, mapred-site.xml and yarn-site.xml as shown below.

5. Create the NameNode and DataNode storage directories and ensure the current user has permission to access them.

6. Format the NameNode once using **hdfs namenode -format**.

7. Start HDFS using start-dfs.sh and YARN using start-yarn.sh. Check running Java processes with jps.

8. Open the NameNode web interface, usually http://localhost:9870, to verify the HDFS service.

Configuration files

Example settings for a single-machine pseudo-distributed lab setup:

```
# ~/.bashrc
export JAVA_HOME=/usr/lib/jvm/java-11-openjdk-amd64
export HADOOP_HOME=/usr/local/hadoop
export PATH=$PATH:$HADOOP_HOME/bin:$HADOOP_HOME/sbin
```

```
<!-- $HADOOP_HOME/etc/hadoop/core-site.xml -->
<configuration>
  <property>
    <name>fs.defaultFS</name>
    <value>hdfs://localhost:9000</value>
  </property>
</configuration>
```

```
<!-- $HADOOP_HOME/etc/hadoop/hdfs-site.xml -->
<configuration>
  <property>
    <name>dfs.replication</name>
    <value>1</value>
  </property>
  <property>
    <name>dfs.namenode.name.dir</name>
    <value>file:///usr/local/hadoop/data/namenode</value>
  </property>
  <property>
    <name>dfs.datanode.data.dir</name>
    <value>file:///usr/local/hadoop/data/datanode</value>
  </property>
</configuration>
```

```
<!-- $HADOOP_HOME/etc/hadoop/mapred-site.xml -->
<configuration>
  <property>
    <name>mapreduce.framework.name</name>
    <value>yarn</value>
  </property>
</configuration>
```

```
<!-- $HADOOP_HOME/etc/hadoop/yarn-site.xml -->
<configuration>
  <property>
    <name>yarn.nodemanager.aux-services</name>
    <value>mapreduce_shuffle</value>
```

```
  </property>
</configuration>
```

Startup scripts / commands

```
source ~/.bashrc
hdfs namenode -format       # Run only for first-time initialization
start-dfs.sh
start-yarn.sh
jps
# To stop the services:
stop-yarn.sh
stop-dfs.sh
```

Output of Program

```
$ jps
1234 NameNode
1350 DataNode
1480 SecondaryNameNode
1590 ResourceManager
1702 NodeManager
1800 Jps
```

Result

Hadoop is installed and configured in pseudo-distributed mode. HDFS and YARN daemons are started and verified using jps and the NameNode web interface.

QUESTION 2: Java program for matrix multiplication using Hadoop MapReduce

Aim

To multiply two matrices using Hadoop MapReduce and display the resultant matrix.

Procedure

1. Create two input files, matrixA.txt and matrixB.txt. Each record contains matrix name, row index, column index and value.

2. Compile the Java program with the Hadoop classpath and package the classes into a JAR file.

3. Copy the input directory to HDFS and execute the MapReduce job.

4. The mapper emits each A(i,k) value for every output column and each B(k,j) value for every output row.

5. The reducer groups values by output cell (i,j), multiplies matching A(i,k) and B(k,j) values, and sums the products.

6. Read the output directory from HDFS.

Input format

```
A,0,0,1
A,0,1,2
A,1,0,3
A,1,1,4
B,0,0,5
B,0,1,6
B,1,0,7
B,1,1,8
```

Program

```
import java.io.IOException;
import java.util.*;
import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.fs.Path;
import org.apache.hadoop.io.*;
import org.apache.hadoop.mapreduce.*;
import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;

public class MatrixMultiply {
  public static class M extends Mapper<LongWritable,Text,Text,Text> {
    public void map(LongWritable key, Text value, Context c)
        throws IOException, InterruptedException {
```

```
      String[] x = value.toString().trim().split(",");
      String matrix = x[0];
      int r = Integer.parseInt(x[1]);
      int col = Integer.parseInt(x[2]);
      int v = Integer.parseInt(x[3]);
      int n = c.getConfiguration().getInt("matrix.size", 2);
      if (matrix.equals("A")) {
        for (int j=0; j<n; j++)
          c.write(new Text(r+","+j), new Text("A,"+col+","+v));
      } else {
        for (int i=0; i<n; i++)
          c.write(new Text(i+","+col), new Text("B,"+r+","+v));
      }
    }
  }

  public static class R extends Reducer<Text,Text,Text,IntWritable> {
    public void reduce(Text key, Iterable<Text> values, Context c)
        throws IOException, InterruptedException {
      Map<Integer,Integer> a = new HashMap<>();
      Map<Integer,Integer> b = new HashMap<>();
      for (Text t : values) {
        String[] x = t.toString().split(",");
        int k = Integer.parseInt(x[1]);
        int v = Integer.parseInt(x[2]);
        if (x[0].equals("A")) a.put(k,v);
        else b.put(k,v);
      }
      int sum = 0;
      for (Integer k : a.keySet())
        if (b.containsKey(k)) sum += a.get(k) * b.get(k);
      c.write(key, new IntWritable(sum));
    }
  }

  public static void main(String[] args) throws Exception {
    Configuration conf = new Configuration();
    conf.setInt("matrix.size", 2); // Change for other square matrix sizes
    Job job = Job.getInstance(conf, "Matrix Multiplication");
    job.setJarByClass(MatrixMultiply.class);
    job.setMapperClass(M.class);
    job.setReducerClass(R.class);
    job.setOutputKeyClass(Text.class);
    job.setOutputValueClass(Text.class);
    job.setMapOutputKeyClass(Text.class);
    job.setMapOutputValueClass(Text.class);
    FileInputFormat.addInputPath(job, new Path(args[0]));
    FileOutputFormat.setOutputPath(job, new Path(args[1]));
    System.exit(job.waitForCompletion(true) ? 0 : 1);
  }
}
```

Compile and execute

```
export HADOOP_CLASSPATH=$(hadoop classpath)
javac -classpath "$HADOOP_CLASSPATH" -d classes MatrixMultiply.java
jar -cvf matrixmultiply.jar -C classes .
hdfs dfs -mkdir -p /input/matrix
hdfs dfs -put matrixA.txt /input/matrix/
hdfs dfs -put matrixB.txt /input/matrix/
hadoop jar matrixmultiply.jar MatrixMultiply /input/matrix /output/matrix
hdfs dfs -cat /output/matrix/part-r-00000
```

Output of Program

```
0,0    19
0,1    22
1,0    43
1,1    50
```

Result

The MapReduce program successfully multiplies the two 2×2 matrices and produces the resultant matrix [[19, 22], [43, 50]].

QUESTION 3: HDFS file management: adding files and directories

Aim

To create directories in HDFS and add local files and directories to HDFS.

Procedure

1. Start Hadoop HDFS services.

2. Create a directory in HDFS using hdfs dfs -mkdir.

3. Copy a local file into HDFS using -put or -copyFromLocal.

4. Copy a complete local directory into HDFS using -put.

5. List the HDFS directory and verify the uploaded files.

Commands

```
start-dfs.sh
hdfs dfs -mkdir -p /user/student/bigdata
hdfs dfs -put sample.txt /user/student/bigdata/
hdfs dfs -put local_folder /user/student/bigdata/
hdfs dfs -ls -R /user/student/bigdata
hdfs dfs -cat /user/student/bigdata/sample.txt
```

Output of Program

```
Found 2 items
/user/student/bigdata/local_folder
/user/student/bigdata/sample.txt
Contents of sample.txt:
Welcome to Hadoop HDFS
```

Result

The HDFS directory was created and the local file and directory were successfully added and verified.

QUESTION 4: Java program to count the number of characters in each word using Hadoop MapReduce

Aim

To count the number of characters present in each word in an input file using Hadoop MapReduce.

Procedure

1. Create an input text file containing words.

2. The mapper reads each line, splits it into words, and emits the word as key and its character count as value.

3. The reducer receives each word and its count and writes the result.

4. Compile the Java program, create a JAR file, upload the input to HDFS and run the job.

5. Display the output using hdfs dfs -cat.

Program

```
import java.io.IOException;
import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.fs.Path;
import org.apache.hadoop.io.*;
import org.apache.hadoop.mapreduce.*;
import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;

public class WordCharacterCount {
  public static class M extends Mapper<LongWritable,Text,Text,IntWritable> {
    public void map(LongWritable key, Text value, Context c)
        throws IOException, InterruptedException {
      String line = value.toString().toLowerCase();
      for (String word : line.split("\\W+")) {
        if (!word.isEmpty())
          c.write(new Text(word), new IntWritable(word.length()));
      }
    }
  }
  public static class R extends Reducer<Text,IntWritable,Text,IntWritable> {
    public void reduce(Text word, Iterable<IntWritable> counts, Context c)
        throws IOException, InterruptedException {
      // All occurrences of the same word have the same character length.
```

```
      int length = 0;
      for (IntWritable x : counts) length = x.get();
      c.write(word, new IntWritable(length));
    }
  }
  public static void main(String[] args) throws Exception {
    Job job = Job.getInstance(new Configuration(), "Characters in Each Word");
    job.setJarByClass(WordCharacterCount.class);
    job.setMapperClass(M.class);
    job.setReducerClass(R.class);
    job.setOutputKeyClass(Text.class);
    job.setOutputValueClass(IntWritable.class);
    FileInputFormat.addInputPath(job, new Path(args[0]));
    FileOutputFormat.setOutputPath(job, new Path(args[1]));
    System.exit(job.waitForCompletion(true) ? 0 : 1);
  }
}
```

Compile and execute

```
export HADOOP_CLASSPATH=$(hadoop classpath)
javac -classpath "$HADOOP_CLASSPATH" -d classes WordCharacterCount.java
jar -cvf wordchars.jar -C classes .
hdfs dfs -mkdir -p /input/words
hdfs dfs -put words.txt /input/words/
hadoop jar wordchars.jar WordCharacterCount /input/words /output/wordchars
hdfs dfs -cat /output/wordchars/part-r-00000
```

Output of Program

```
bigdata    7
hello      5
hadoop     6
world      5
```

Result

The MapReduce program successfully displays the character length of each distinct word. Repeated occurrences of a word are grouped under the same key.

QUESTION 5: HDFS file management: adding files to HDFS

Aim

To transfer a file from the local file system to the Hadoop Distributed File System.

Procedure

1. Start HDFS.

2. Create the destination directory in HDFS.

3. Use the -put command to upload the local file.

4. Use -ls to confirm the file exists and -cat to display its contents.

Commands

```
start-dfs.sh
hdfs dfs -mkdir -p /user/student/input
hdfs dfs -put data.txt /user/student/input/
hdfs dfs -ls /user/student/input
hdfs dfs -cat /user/student/input/data.txt
```

Output of Program

```
Found 1 items
-rw-r--r--   1 student supergroup   25  data.txt
Sample data for HDFS
```

Result

The local file data.txt was successfully copied to the specified HDFS directory.

QUESTION 6: Java program to count the number of words using Hadoop MapReduce

Aim

To count the occurrences of each word in a text file using Hadoop MapReduce.

Procedure

1. Prepare a text file containing sample sentences and upload it to HDFS.

2. The mapper converts each line to lowercase, splits it into words and emits (word, 1).

3. The framework groups values by word.

4. The reducer adds all values for each word and emits (word, total count).

5. Compile, package and run the program; display the reducer output from HDFS.

Program

```
import java.io.IOException;
import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.fs.Path;
import org.apache.hadoop.io.*;
import org.apache.hadoop.mapreduce.*;
import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;

public class WordCount {
  public static class M extends Mapper<LongWritable,Text,Text,IntWritable> {
    private static final IntWritable ONE = new IntWritable(1);
    public void map(LongWritable key, Text value, Context c)
        throws IOException, InterruptedException {
      String line = value.toString().toLowerCase();
      for (String word : line.split("\\W+"))
        if (!word.isEmpty()) c.write(new Text(word), ONE);
    }
  }
  public static class R extends Reducer<Text,IntWritable,Text,IntWritable> {
    public void reduce(Text word, Iterable<IntWritable> values, Context c)
        throws IOException, InterruptedException {
      int sum = 0;
      for (IntWritable x : values) sum += x.get();
      c.write(word, new IntWritable(sum));
    }
  }
  public static void main(String[] args) throws Exception {
    Job job = Job.getInstance(new Configuration(), "Word Count");
    job.setJarByClass(WordCount.class);
    job.setMapperClass(M.class);
    job.setReducerClass(R.class);
    job.setOutputKeyClass(Text.class);
    job.setOutputValueClass(IntWritable.class);
    FileInputFormat.addInputPath(job, new Path(args[0]));
    FileOutputFormat.setOutputPath(job, new Path(args[1]));
    System.exit(job.waitForCompletion(true) ? 0 : 1);
  }
}
```

Compile and execute

```
export HADOOP_CLASSPATH=$(hadoop classpath)
javac -classpath "$HADOOP_CLASSPATH" -d classes WordCount.java
jar -cvf wordcount.jar -C classes .
hdfs dfs -mkdir -p /input/wc
hdfs dfs -put input.txt /input/wc/
hadoop jar wordcount.jar WordCount /input/wc /output/wc
hdfs dfs -cat /output/wc/part-r-00000
```

Output of Program

```
bigdata    2
hadoop     2
hello      1
world      1
```

Result

The Hadoop MapReduce job successfully calculates the frequency of every word in the input file.

QUESTION 7: HDFS file management: deleting files

Aim

To delete a file or directory from HDFS using Hadoop file system commands.

Procedure

1. Start HDFS and list the target directory.

2. Delete a file using hdfs dfs -rm.

3. Delete a directory and its contents recursively using hdfs dfs -rm -r.

4. List the parent directory again to verify deletion.

Commands

```
start-dfs.sh
hdfs dfs -ls /user/student/input
hdfs dfs -rm /user/student/input/data.txt
# Delete a directory and all its contents:
hdfs dfs -rm -r /user/student/old_directory
hdfs dfs -ls /user/student/input
```

Output of Program

```
Deleted /user/student/input/data.txt
# The deleted file is no longer listed.
```

Result

The specified file and directory were successfully removed from HDFS.

QUESTION 8: Downloading and installing Hive; startup scripts

Aim

To install Apache Hive, configure its environment and start the Hive services for executing HiveQL queries.

Procedure

1. Install and configure Hadoop first, and ensure HDFS and YARN are working.

2. Install a Java version supported by the chosen Hive release. Download the Apache Hive binary distribution and extract it, for example, to /usr/local/hive.

3. Set HIVE_HOME and add its bin directory to PATH.

4. Configure hive-site.xml with the metastore connection and warehouse directory. For a lab installation, Derby can be used as an embedded metastore; a production setup should use a supported external database.

5. Create the warehouse directory in HDFS and grant suitable permissions.

6. Initialize the metastore schema once, then start Hive using the Hive CLI or Beeline with the appropriate service.

7. Run a simple SHOW DATABASES query to verify the installation.

Environment and configuration

```
# ~/.bashrc
export HIVE_HOME=/usr/local/hive
export PATH=$PATH:$HIVE_HOME/bin
export HADOOP_HOME=/usr/local/hadoop
```

```
<!-- $HIVE_HOME/conf/hive-site.xml (illustrative embedded Derby setup) -->
<configuration>
  <property>
    <name>javax.jdo.option.ConnectionURL</name>
    <value>jdbc:derby:;databaseName=metastore_db;create=true</value>
  </property>
  <property>
```

```
    <name>hive.metastore.warehouse.dir</name>
    <value>/user/hive/warehouse</value>
  </property>
</configuration>
```

Startup commands

```
source ~/.bashrc
hdfs dfs -mkdir -p /user/hive/warehouse
hdfs dfs -chmod 777 /user/hive/warehouse
schematool -dbType derby -initSchema   # First-time setup only
hive
# At the hive prompt:
SHOW DATABASES;
# Or use Beeline when HiveServer2 is configured:
beeline -u jdbc:hive2://localhost:10000/default
```

Output of Program

```
hive> SHOW DATABASES;
OK
default
Time taken: 1.2 seconds
```

Result

Hive is installed and initialized. The Hive prompt opens and executes HiveQL commands successfully.

Version note: Hive, Hadoop and Java compatibility varies by release. Use a mutually compatible combination specified by the laboratory; do not assume the illustrative paths or Java version apply to every release.

QUESTION 9: HiveQL: create employees table, load data and display table contents

Aim

To create an employees table in Hive, load records into it and retrieve the table contents.

Procedure

1. Start Hadoop and open the Hive shell or Beeline session.
2. Create a table named employees with the fields eid, ename, eage and edepartment.
3. Prepare a comma-separated text file containing employee records.
4. Load the local data file into the Hive table.
5. Execute SELECT * to display all records.

Program / HiveQL commands

```
CREATE TABLE employees (
  eid INT,
  ename STRING,
  eage INT,
  edepartment STRING
)
ROW FORMAT DELIMITED
FIELDS TERMINATED BY ','
STORED AS TEXTFILE;

-- employees.txt (one record per line)
101,Ravi,24,IT
102,Meena,25,HR
103,Arun,23,Sales

LOAD DATA LOCAL INPATH '/home/student/employees.txt'
INTO TABLE employees;

SELECT * FROM employees;
```

Output of Program

```
+-----------+-------------+------------+----------------+
| eid       | ename       | eage       | edepartment    |
+-----------+-------------+------------+----------------+
| 101       | Ravi        | 24         | IT             |
```

```
| 102       | Meena       | 25         | HR             |
| 103       | Arun        | 23         | Sales          |
+-----------+-------------+------------+----------------+
```

Result

The employees table was created, the sample records were loaded, and all employee details were displayed successfully.

QUESTION 10: Downloading and installing HBase; startup scripts and configuration files

Aim

To install Apache HBase, configure it to use Hadoop HDFS and start the HBase services.

Procedure

1. Install and configure a compatible Java version and Hadoop installation.

2. Download the binary distribution of Apache HBase and extract it, for example, to /usr/local/hbase.

3. Set HBASE_HOME and add its bin directory to PATH.

4. Configure hbase-env.sh with JAVA_HOME. Configure hbase-site.xml with the HDFS root directory and standalone or distributed mode.

5. Ensure HDFS is running before starting HBase when HBase is configured to use HDFS.

6. Start HBase using start-hbase.sh and verify the master and region server processes.

7. Open the HBase web UI, commonly available on port 16010, if enabled.

Configuration

```
# $HBASE_HOME/conf/hbase-env.sh
export JAVA_HOME=/usr/lib/jvm/java-11-openjdk-amd64
```

```
<!-- $HBASE_HOME/conf/hbase-site.xml -->
<configuration>
  <property>
    <name>hbase.rootdir</name>
    <value>hdfs://localhost:9000/hbase</value>
  </property>
  <property>
    <name>hbase.cluster.distributed</name>
    <value>false</value>
  </property>
  <property>
    <name>hbase.unsafe.stream.capability.enforce</name>
    <value>false</value>
  </property>
</configuration>
```

Startup scripts / commands

```
start-dfs.sh
start-hbase.sh
jps
# Stop HBase:
stop-hbase.sh
```

Output of Program

```
$ jps
NameNode
DataNode
HMaster
HRegionServer
Jps
```

Result

HBase is installed and configured. The HBase master and region server are started and verified.

The example uses standalone HBase mode. For a fully distributed cluster, configure the appropriate distributed settings and ZooKeeper services according to the installed HBase release.

QUESTION 11: Installing Thrift, generating HBase Thrift binding and interacting with HBase

Aim

To install Apache Thrift, generate client bindings for the HBase Thrift interface and use a client to interact with HBase.

Procedure

1. Install a Thrift compiler version compatible with the HBase Thrift interface and install the required language runtime, such as Python.

2. Download the HBase source or distribution containing the Thrift interface definition file (Hbase.thrift).

3. Start HBase and start the HBase Thrift service using the script provided by the installed HBase release.

4. Generate client-side bindings from Hbase.thrift using the Thrift compiler.

5. Configure the generated client with the Thrift server hostname and port.

6. Use the generated client to connect to HBase and perform table operations. The exact generated API names can differ between Thrift versions.

Commands

```
# Install Thrift using the package manager or the official
# Apache Thrift installation instructions.
thrift --version

# Locate Hbase.thrift in the HBase distribution/source tree.
# Generate Python bindings (output folder is created if needed):
mkdir -p gen-py
thrift --gen py -out gen-py Hbase.thrift

# Start HBase, then start the Thrift service.
start-hbase.sh
hbase thrift start
# The default Thrift port is commonly 9090.
```

Example Python client (legacy HBase Thrift API)

```
from thrift.transport import TSocket, TTransport
from thrift.protocol import TBinaryProtocol
from hbase import Hbase

socket = TSocket.TSocket('localhost', 9090)
transport = TTransport.TBufferedTransport(socket)
protocol = TBinaryProtocol.TBinaryProtocol(transport)
client = Hbase.Client(protocol)

transport.open()
print(client.getTableNames())
transport.close()
```

Output of Program

```
Connected to HBase Thrift server.
Tables:
['student']
```

Result

Thrift bindings were generated and the client connected to the HBase Thrift service to retrieve the table list.

Important: HBase Thrift APIs have changed across releases. Use the Hbase.thrift file shipped with the exact installed HBase version and its matching generated client. Some releases use a separate hbase thrift start command or service script.

QUESTION 13: Hive DDL commands

Aim

To perform Data Definition Language (DDL) operations in Hive for creating, altering, describing and dropping database objects.

Procedure

1. Open the Hive shell or connect through Beeline.

2. Execute CREATE DATABASE and CREATE TABLE to define database objects.

3. Use DESCRIBE or SHOW commands to inspect the objects.

4. Use ALTER TABLE to modify a table definition.

5. Use DROP TABLE or DROP DATABASE to remove an object when required.

Program / HiveQL commands

```
-- Create a database
CREATE DATABASE college;

-- Select the database
USE college;

-- Create a table
CREATE TABLE student (
  sid INT,
  sname STRING,
  department STRING
)
ROW FORMAT DELIMITED
FIELDS TERMINATED BY ',';

-- Display databases and tables
SHOW DATABASES;
SHOW TABLES;

-- Display table structure
DESCRIBE student;

-- Add a column
ALTER TABLE student ADD COLUMNS (age INT);

-- Rename a table
ALTER TABLE student RENAME TO student_details;

-- Drop a table
DROP TABLE student_details;

-- Drop the database after its tables are removed
DROP DATABASE college;
```

Output of Program

```
OK
Database college created
Table student created
Database selected
Table list displayed
Table structure displayed
Table altered / renamed
Table dropped
Database college dropped
```

Result

The Hive DDL commands were executed to create, inspect, modify and remove database objects successfully.

QUESTION 14: Hive DML commands

Aim

To perform Data Manipulation Language (DML) operations in Hive, such as loading, inserting, selecting and exporting table data.

Procedure

1. Create or select a Hive database and create a table.

2. Load records from a local file or HDFS into the table.

3. Insert additional records using INSERT INTO or overwrite existing table data using INSERT OVERWRITE.

4. Retrieve records using SELECT with filtering and sorting clauses.

5. Export table data to a local path or HDFS location when required.

Program / HiveQL commands

```
CREATE DATABASE IF NOT EXISTS college;
USE college;

CREATE TABLE student (
  sid INT,
  sname STRING,
  department STRING
)
ROW FORMAT DELIMITED
FIELDS TERMINATED BY ',';

-- Load a local file into the table
LOAD DATA LOCAL INPATH '/home/student/student.txt'
INTO TABLE student;

-- Insert a new record
INSERT INTO TABLE student
VALUES (104, 'Kumar', 'CSE');

-- Replace table contents with the query result
INSERT OVERWRITE TABLE student
SELECT sid, sname, department
FROM student
WHERE department = 'CSE';

-- Retrieve all records
SELECT * FROM student;

-- Retrieve selected records
SELECT sname, department
FROM student
WHERE sid >= 102;

-- Export query results to HDFS
INSERT OVERWRITE DIRECTORY '/output/student'
ROW FORMAT DELIMITED
FIELDS TERMINATED BY ','
SELECT * FROM student;
```

Output of Program

```
The LOAD and INSERT statements complete successfully.
SELECT * FROM student;
104    Kumar    CSE

The export operation writes the selected rows to /output/student in HDFS.
```

Result

Hive DML commands were used to load, insert, query and export records from the student table.

The INSERT OVERWRITE example replaces the table contents with the filtered query result. Run it only when replacement is intended.

QUESTION 16: Java program for multiplication of two numbers using Hadoop MapReduce

Aim

To multiply two integer numbers using a Hadoop MapReduce program.

Procedure

1. Create an input file containing the two integer values separated by a comma.

2. The mapper reads the input line, extracts the two numbers, multiplies them and emits the product with a fixed key.

3. The reducer receives the product value and writes it to the output.

4. Compile the Java source file, package it into a JAR and upload the input file to HDFS.

5. Run the Hadoop job and display the output file.

Program

```
import java.io.IOException;
import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.fs.Path;
import org.apache.hadoop.io.*;
import org.apache.hadoop.mapreduce.*;
import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;

public class MultiplyNumbers {
  public static class M extends Mapper<LongWritable,Text,Text,IntWritable> {
    public void map(LongWritable key, Text value, Context c)
        throws IOException, InterruptedException {
      String[] n = value.toString().trim().split(",");
      int a = Integer.parseInt(n[0].trim());
      int b = Integer.parseInt(n[1].trim());
      c.write(new Text("Product"), new IntWritable(a * b));
    }
  }
  public static class R extends Reducer<Text,IntWritable,Text,IntWritable> {
    public void reduce(Text key, Iterable<IntWritable> values, Context c)
        throws IOException, InterruptedException {
      for (IntWritable v : values)
        c.write(key, new IntWritable(v.get()));
    }
  }
  public static void main(String[] args) throws Exception {
    Job job = Job.getInstance(new Configuration(), "Multiply Two Numbers");
    job.setJarByClass(MultiplyNumbers.class);
    job.setMapperClass(M.class);
    job.setReducerClass(R.class);
    job.setOutputKeyClass(Text.class);
    job.setOutputValueClass(IntWritable.class);
    FileInputFormat.addInputPath(job, new Path(args[0]));
    FileOutputFormat.setOutputPath(job, new Path(args[1]));
    System.exit(job.waitForCompletion(true) ? 0 : 1);
  }
}
```

Input file

```
# numbers.txt
10,20
```

Compile and execute

```
export HADOOP_CLASSPATH=$(hadoop classpath)
javac -classpath "$HADOOP_CLASSPATH" -d classes MultiplyNumbers.java
jar -cvf multiply.jar -C classes .
hdfs dfs -mkdir -p /input/multiply
hdfs dfs -put numbers.txt /input/multiply/
hadoop jar multiply.jar MultiplyNumbers /input/multiply /output/multiply
hdfs dfs -cat /output/multiply/part-r-00000
```

Output of Program

```
Product    200
```

Result

The Hadoop MapReduce program successfully multiplies 10 and 20 and displays the product 200.

LAB TEST EXECUTION CHECKLIST

• Start HDFS before running HDFS, Hive or HBase operations.

• Use a new output directory for every MapReduce run; Hadoop normally fails if the output directory already exists.

• Before rerunning a job, remove its old output directory with hdfs dfs -rm -r /output/jobname.

• Use the exact Java, Hadoop, Hive and HBase versions installed in the lab; their compatibility matters.

• Use hdfs dfs -ls and hdfs dfs -cat to verify HDFS files and MapReduce output.

• For Java MapReduce programs, compile with the Hadoop classpath and include the correct main class in the hadoop jar command.

• Sample output shown in this guide is expected output for the sample input, not a live execution log.

Common MapReduce command pattern

```
export HADOOP_CLASSPATH=$(hadoop classpath)
javac -classpath "$HADOOP_CLASSPATH" -d classes ProgramName.java
jar -cvf program.jar -C classes .
hdfs dfs -rm -r -f /output/result
hadoop jar program.jar MainClass /input/path /output/result
hdfs dfs -cat /output/result/part-r-00000
```

Prepared from the uploaded CCS334 – Big Data Analytics (Regulation 2021) question paper. The paper lists Questions 1–17; Questions 12, 15 and 17 have intentionally been excluded as requested.